

PROYECTO SEMESTRAL

Desarrollo de Software
NRC 25783
02-05-2026

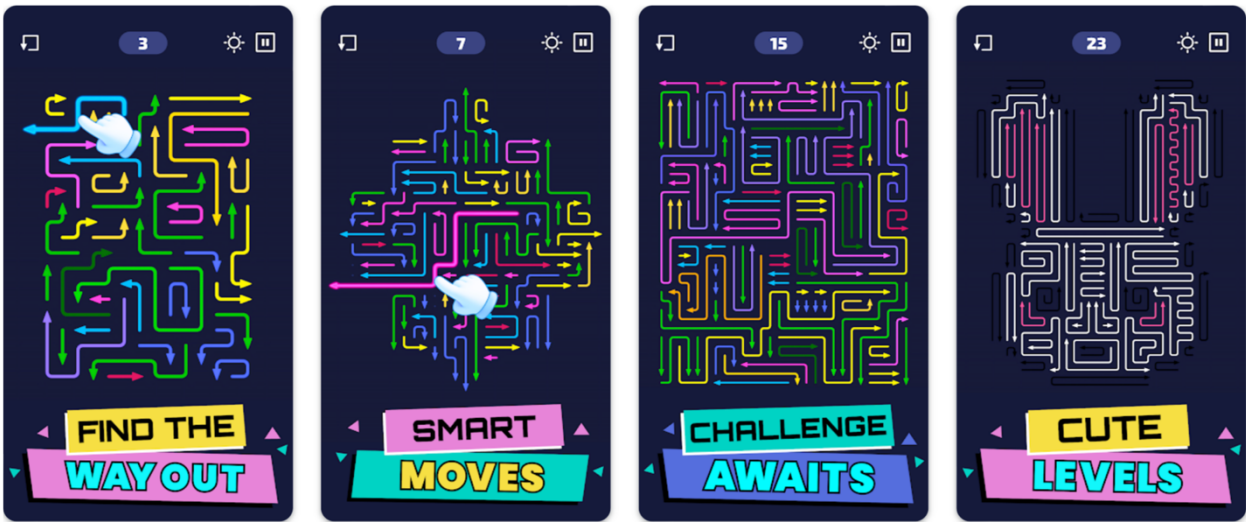
Desarrollo de un clon del juego casual

Arrow Maze — Escape Puzzle

Asignatura	Desarrollo de Software
Modalidad	Grupos de 3 estudiantes
Referencia	Arrow Maze - Escape Puzzle · SayGames Ltd. · Google Play
Tecnología	A definir por el equipo (Flutter, React Native, Unity, Godot, etc.)
Valor total	20 puntos (10% de la nota de la práctica)
Fecha de entrega	Viernes 03 de julio de 2026
Uso de IA	PERMITIDO con documentación obligatoria (ver Sección 7)

1. Descripción del Juego de Referencia

Arrow Maze — Escape Puzzle es un juego casual desarrollado por SayGames Ltd., disponible en Google Play (más de 1 000 000 de descargas, calificación 4.7/5). Ver detalles en el link [Juego Arrow Maze](#).



1.1 Mecánicas principales

- Tablero en cuadrícula (grid) con celdas que contienen flechas en cuatro direcciones (arriba, abajo, izquierda, derecha).
- El jugador toca una celda para rotar o activar la dirección de su flecha.

- Dificultad progresiva: niveles fácil, medio y difícil.
- Elementos adicionales: paredes, celdas vacías, coleccionables y tiempo límite en niveles avanzados.
- Sistema de puntuación basado en número de movimientos y tiempo empleado.
- Pantallas: inicio, selección de nivel, juego, victoria y derrota.

2. Descripción del Proyecto

El objetivo de este proyecto semestral es que los estudiantes, organizados en grupos de 3-4 personas, diseñen e implementen un clon funcional del juego Arrow Maze aplicando los principios y buenas prácticas de ingeniería de software abordados durante el curso. El proyecto contempla dos repositorios independientes:

- Repositorio del Juego (cliente): aplicación móvil que implementa las mecánicas del juego.
- Repositorio del Backend: API REST o servicio web que gestiona usuarios, puntuaciones, niveles y progreso de juego.

Ambos repositorios deben estar alojados en GitHub, con ramas protegidas, pull requests y un historial de commits limpio y semántico. Los README de ambos repositorios deben estar redactados siguiendo las mejores prácticas de la industria (ver Sección 6).

3. Requisitos Técnicos Obligatorios

3.1 Principios SOLID

Todo el código del proyecto debe evidenciar la aplicación de los principios SOLID. Cada principio usado debe estar documentado en el README correspondiente con al menos un ejemplo concreto extraído del código del proyecto, indicando la clase o módulo donde se aplica y la justificación de su uso.

- S — Single Responsibility Principle (SRP): cada clase o módulo tiene una única razón de cambio. Ejemplo: separar la lógica de movimiento (`PlayerMovementService`) de la lógica de renderizado (`PlayerRenderer`) y de la lógica de persistencia (`PlayerRepository`).
- O — Open/Closed Principle (OCP): las entidades están abiertas a la extensión pero cerradas a la modificación. Ejemplo: implementar nuevos tipos de celda o power-ups mediante nuevas clases que extiendan una interfaz `ICell`, sin alterar el código existente.
- L — Liskov Substitution Principle (LSP): cualquier subclase puede sustituir a su clase padre sin alterar el comportamiento del programa. Ejemplo: `WallCell` y `ArrowCell` deben ser intercambiables donde se espere una `ICell`.
- I — Interface Segregation Principle (ISP): ninguna clase debe depender de métodos que no utiliza. Ejemplo: separar `IRenderable`, `ICollidable` e `IInteractable` en interfaces distintas en lugar de agruparlas en una sola interfaz grande.
- D — Dependency Inversion Principle (DIP): los módulos de alto nivel no dependen de los de bajo nivel; ambos dependen de abstracciones. Ejemplo: los casos de uso dependen de interfaces de repositorio (`ILevelRepository`), no de implementaciones concretas de base de datos.

3.2 Patrones de Diseño (GoF)

El proyecto debe implementar varios patrones de diseño de la clasificación Gang of Four, distribuidos entre las tres categorías. Para cada patrón, el equipo debe justificar su elección en el contexto del juego, mostrar un fragmento de código representativo y documentarlo en el README.

Patrones Creacionales

- **Factory Method / Abstract Factory:** creación de diferentes tipos de celdas del tablero (ArrowCell, WallCell, EmptyCell, ExitCell) mediante una fábrica, evitando instanciar clases concretas desde el código cliente.
- **Builder:** construcción de niveles complejos a partir de archivos de configuración JSON o YAML, permitiendo ensamblar paso a paso el tablero, las reglas y los elementos opcionales.
- **Singleton:** gestores globales de ciclo de vida único, como AudioManager, LevelConfigManager o SessionManager, garantizando una sola instancia durante la ejecución.

Patrones Estructurales

- **Composite:** representar el tablero como una estructura jerárquica de BoardGroup y Cell, permitiendo tratar grupos de celdas y celdas individuales de forma uniforme.
- **Decorator:** agregar comportamientos adicionales a las celdas en tiempo de ejecución, por ejemplo LockedCellDecorator o CollectableCellDecorator, sin modificar la celda base.
- **Adapter:** envolver librerías externas de base de datos, análisis de eventos o redes para que cumplan las interfaces internas del proyecto, aislando el código de negocio de dependencias externas.
- **Facade:** proveer una interfaz simplificada para el subsistema de audio, persistencia o red (GameServiceFacade), ocultando la complejidad interna a las capas superiores.

Patrones de Comportamiento

- **Strategy:** intercambiar en tiempo de ejecución el algoritmo de generación de niveles (RandomLevelStrategy, FileBasedLevelStrategy) o la estrategia de puntuación.
- **Observer:** notificar automáticamente a los oyentes (UI, sistema de puntuación, audio) cuando ocurren eventos del juego como LevelCompleted, PlayerMoved o ScoreUpdated.
- **Command:** encapsular cada movimiento del jugador como un objeto Command para soportar historial de acciones y funcionalidad de deshacer/rehacer.
- **State:** gestionar el ciclo de vida del juego mediante estados explícitos: MenuState, PlayingState, PausedState, GameOverState, VictoryState.
- **Template Method:** definir el esqueleto del flujo de un nivel (inicializar, ejecutar, evaluar resultado) en una clase abstracta BaseLevel, delegando pasos específicos a subclasses.

3.3 Arquitectura CLEAN

El proyecto debe implementarse siguiendo la Clean Architecture propuesta por Robert C. Martin (Uncle Bob). Esta arquitectura organiza el sistema en capas concéntricas donde la regla de dependencia establece que siempre se apunta hacia adentro (hacia el dominio), nunca hacia afuera.

Regla fundamental: ninguna capa interna conoce nada acerca de las capas externas. Los detalles de frameworks, bases de datos e interfaces de usuario son **decisiones aplazables**.

Capa 1 — Entidades (Entities / Domain)

Contiene los objetos de negocio fundamentales del juego, independientes de cualquier framework o librería externa. Son clases puras que solo dependen del lenguaje de programación:

- Sin dependencias externas. Pueden ser probadas en aislamiento total.
- Esta capa no importa nada de las capas exteriores.

Capa 2 — Casos de Uso (Use Cases / Application Layer)

Contiene la lógica de aplicación del juego. Orquesta el flujo de datos entre entidades y define qué hace el sistema.

- Dependen únicamente de interfaces (puertos) definidas en esta misma capa, nunca de implementaciones concretas.

Capa 3 — Adaptadores de Interfaz (Interface Adapters)

Convierte datos entre el formato del dominio y el formato de frameworks externos. Contiene:

- Presenters, ViewModels y Controllers para cada pantalla del juego.
- Implementaciones de los repositorios (adaptan la capa de datos al puerto definido en los casos de uso).
- Mapeadores (Mappers) que transforman entidades de dominio en DTOs y viceversa.

Capa 4 — Frameworks e Infraestructura (Frameworks & Drivers)

Contiene los detalles de implementación más volátiles y reemplazables:

- Motor de juego o framework de UI (Flutter, React Native, Unity, Godot).
- Base de datos local (SQLite, Hive, Room) y base de datos remota (PostgreSQL, Firebase).
- Librerías de red, servicios de autenticación y APIs externas.
- Esta capa puede reemplazarse sin afectar las capas internas.

Entregable obligatorio: Diagrama de Arquitectura CLEAN

El equipo debe entregar un diagrama que muestre claramente las cuatro capas concéntricas y las dependencias entre ellas, identificando en qué capa se ubica cada componente principal del juego.

- El diagrama debe generarse con draw.io, PlantUML, Lucidchart o herramienta equivalente.
- Debe incluirse como imagen en el README y como archivo fuente editable en el repositorio.
- Las dependencias entre capas deben respetar estrictamente la regla de dependencia (de afuera hacia adentro).

3.4 Programación Orientada a Aspectos (AOP)

El equipo debe implementar Programación Orientada a Aspectos (AOP) para separar las responsabilidades transversales (***cross-cutting concerns***) del código de negocio, evitando que la lógica de infraestructura contamine las capas de dominio y aplicación. Se deben implementar un mínimo de un (1) aspecto:

- **Logging y Trazabilidad:** registrar automáticamente la entrada, salida y duración de los métodos críticos de los casos de uso sin introducir llamadas al logger en el código de negocio. Ejemplo: interceptar `MovePlayerUseCase.execute()` para registrar el estado del tablero antes y después del movimiento.
- **Manejo Centralizado de Excepciones:** capturar y gestionar excepciones de operaciones de red o persistencia de forma centralizada, aplicando reintentos o fallbacks sin que el caso de uso conozca este mecanismo.
- **Métricas de Rendimiento / Profiling:** medir el tiempo de ejecución de operaciones costosas (carga de nivel, resolución del camino) para detectar cuellos de botella sin instrumentar manualmente cada método.
- **Seguridad y Autorización:** verificar automáticamente que el usuario tenga sesión activa antes de ejecutar casos de uso que requieran autenticación, como guardar progreso o acceder al ranking global.
- **Caché de Resultados:** aplicar memorización automática a consultas costosas como `GetLeaderboardUseCase` cuando los datos no han cambiado en un intervalo de tiempo.

La implementación puede realizarse **sin hacer uso de librerías AOP**. Se debe implementar AOP haciendo uso de la estrategia SOLID vista en clases y se debe documentar la estrategia en el README.

3.5 Casos de Prueba — Mejores Prácticas de la Industria

El proyecto debe incluir una suite de pruebas completa y automatizada. Se deben aplicar las siguientes categorías de pruebas:

Pruebas Unitarias

- Probar cada entidad, caso de uso y servicio de forma aislada.
- Usar mocks o stubs para sustituir las dependencias externas (repositorios, servicios de red).
- Seguir el patrón AAA (Arrange – Act – Assert) en cada prueba.
- Nombrar las pruebas con la convención: `should_[resultado_esperado]_when_[condicion]`.
- Ejemplo: `should_return_victory_state_when_player_reaches_exit_cell`.
- Las pruebas no deben ser frágiles, deben probar comportamientos, no detalles de implementación.
- Puede hacer uso de las buenas prácticas vistas en clases.

Pruebas de Integración

- Verificar la interacción correcta entre casos de uso y repositorios reales (usando base de datos en memoria).
- Probar los endpoints de la API REST del backend con clientes HTTP de prueba.
- Validar que los flujos completos de guardar y recuperar progreso funcionan de extremo a extremo.

Pruebas de Widget / UI (solo aplicación)

- Verificar que los componentes visuales del juego se renderizan correctamente y responden a interacciones del usuario.
- Probar la navegación entre pantallas (inicio, nivel, victoria, derrota).

Pruebas de Contrato (recomendadas)

- Usar Pact u herramienta equivalente para verificar que el contrato entre el cliente del juego y el backend se respeta ante cambios. (ver link <https://docs.pact.io/>)

Estándares adicionales de calidad

- Integrar las pruebas en el pipeline de CI/CD (GitHub Actions o equivalente): las pruebas deben ejecutarse automáticamente en cada pull request.

4. Diagramas Obligatorios

El equipo debe entregar los siguientes diagramas. Cada uno debe incluirse como imagen de alta resolución en el README del repositorio correspondiente y como archivo fuente editable (.drawio, .puml, etc.) en una carpeta /docs del repositorio.

4.1 Diagrama de Clases

- Debe mostrar todas las clases principales del proyecto (entidades, casos de uso, repositorios, servicios, presenters).
- Incluir relaciones de herencia, implementación de interfaces, asociación, agregación y composición.
- Indicar claramente a qué capa de la Clean Architecture pertenece cada clase (puede usarse color o estereotipo).
- Mostrar los patrones de diseño implementados.
- No es necesario incluir clases de UI de bajo nivel (widgets individuales); enfocarse en la lógica de negocio.

4.2 Diagrama de Capas — Arquitectura CLEAN

- Representación visual de las cuatro capas concéntricas con sus componentes principales.
- Flechas de dependencia que ilustren que las dependencias siempre apuntan hacia el centro. Es decir, se cumple la Regla de Dependencia.
- Leyenda con la correspondencia entre capas y colores.
- Identificar los puertos (interfaces) y adaptadores entre capas.

5. Funcionalidades Mínimas del Juego

El clon debe implementar las siguientes funcionalidades como **requisito mínimo** para la aprobación del proyecto:

5.1 Aplicación (Juego)

1. Pantalla de inicio con nombre del juego, botón de jugar y acceso a ajustes.
2. Pantalla de selección de niveles con indicador de progreso y niveles bloqueados.
3. Motor de juego funcional: tablero en cuadrícula, flechas rotas, movimientos.
4. Al menos 15 niveles diseñados manualmente con dificultad progresiva con diferentes formas de tableros.

5. Sistema de puntuación basado en criterios que debe definir.
6. Pantalla de victoria con puntuación obtenida y opción de siguiente nivel.
7. Pantalla de derrota con opción de reintentar.
8. Persistencia local del progreso del jugador (niveles completados y puntuaciones).
9. Efectos de sonido y música de fondo con opción de silenciar.
10. Soporte para al menos dos idiomas (mínimo: español e inglés).

5.2 Backend (API REST)

1. Autenticación de usuarios (registro e inicio de sesión) con JWT.
2. Endpoint para sincronizar el progreso del jugador con el servidor.
3. Tabla de clasificación global (leaderboard) con los mejores puntajes por nivel.
4. Endpoint para obtener y actualizar la definición de niveles (permite añadir niveles sin actualizar la app).
5. Documentación de la API con Swagger / OpenAPI.
6. Manejo adecuado de errores HTTP y respuestas consistentes.

6. README y Documentación de Repositorios

Ambos repositorios deben contar con un README.md redactado en inglés, claro, completo y profesional, siguiendo las mejores prácticas de la industria. A continuación se detalla el contenido mínimo obligatorio para cada sección.

6.1 Estructura mínima del README (ambos repositorios)

- *Project Title & Badges*: nombre del proyecto, badges de estado del build (CI/CD), tipos de pruebas y licencia.
- *Description*: descripción concisa del proyecto (qué es, para qué sirve, qué tecnologías usa).
- *Demo / Screenshots*: capturas de pantalla o GIF animado del juego en funcionamiento (solo repositorio del juego).
- *Architecture*: explicación de la Clean Architecture aplicada con el diagrama de capas embebido.
- *Design Patterns*: tabla o lista de los patrones implementados con descripción breve y enlace al archivo de código.
- *SOLID Principles*: sección explicando cómo se aplica cada principio con ejemplo de código.
- *AOP*: descripción de los aspectos implementados y la estrategia usada basada en SOLID.
- *Getting Started*: requisitos previos, instrucciones de instalación paso a paso, y comando para ejecutar el proyecto localmente.
- *Running Tests*: comando para ejecutar las pruebas.
- *AI Usage Documentation*: sección obligatoria que documenta el uso de herramientas de inteligencia artificial (ver Sección 7).
- *Contributing*: guía de convenciones de commits (Conventional Commits), flujo de trabajo con branches y proceso de pull request.
- *License*: tipo de licencia del proyecto.

6.2 Convenciones de commits (Conventional Commits)

El historial de Git debe seguir el estándar Conventional Commits. Ejemplos:

- feat(board): add arrow rotation logic
- fix(player): correct movement when hitting wall
- test(use-case): add unit tests for MovePlayerUseCase
- docs(readme): update architecture diagram
- refactor(level): apply Factory Method pattern to cell creation

Los mensajes de commit en inglés son obligatorios. Se recomienda configurar un hook de pre-commit con commitlint para validar el formato automáticamente.

7. Uso de Herramientas de Inteligencia Artificial

El uso de IA está PERMITIDO y es bienvenido en este proyecto

Se permite el uso de herramientas de inteligencia artificial generativa (Claude, ChatGPT, Gemini, GitHub Copilot, Cursor, etc.) para acelerar y mejorar el desarrollo del proyecto. El uso de IA no es un atajo, sino una competencia profesional moderna que los estudiantes deben dominar.

Sin embargo, su uso es condicional a una documentación rigurosa y transparente tal como lo exigen las mejores prácticas de la industria.

7.1 Principio fundamental

El equipo es responsable de todo el código entregado, independientemente de si fue generado con o sin asistencia de IA. No se acepta como justificación de errores, deuda técnica o incumplimiento de los requisitos el hecho de que el código fue generado por una IA. El equipo debe comprender, revisar, validar y ser capaz de explicar cualquier fragmento de código del proyecto.

7.2 Documentación obligatoria del uso de IA

Cada repositorio debe incluir un archivo AI_USAGE.md en la raíz, con el siguiente contenido mínimo:

Herramientas utilizadas

- Nombre de cada herramienta de IA utilizada (p. ej.: Claude 3.7 Sonnet, ChatGPT-4o, GitHub Copilot).
- Versión o modelo específico cuando sea posible identificarla.
- Rol asignado a cada herramienta dentro del flujo de trabajo del equipo.

Registro de uso por tarea

Por cada uso significativo de IA se debe registrar una entrada con:

- Tarea o problema abordado.
- Herramienta de IA utilizada.
- Prompt o instrucción proporcionada (transcripción literal o paráfrasis fiel).
- Resultado obtenido (fragmento de código, diseño, explicación).
- Modificaciones realizadas por el equipo al resultado de la IA.
- Lecciones aprendidas o limitaciones identificadas.

Evaluación crítica

- Porcentaje aproximado del código que contó con asistencia de IA.
- Casos donde la IA produjo resultados incorrectos o subóptimos y cómo se detectaron y corrigieron.
- Reflexión del equipo sobre el impacto de la IA en la productividad y calidad del código.

7.3 Mejores prácticas de uso de IA en desarrollo de software

El equipo debe demostrar que ha utilizado la IA de acuerdo con las mejores prácticas actuales de la industria:

11. **Prompt engineering efectivo:** usar prompts claros, con contexto suficiente, restricciones explícitas y ejemplos cuando sea necesario. Documentar los prompts que produjeron los mejores resultados.
12. **Revisión crítica del output:** nunca aceptar código generado por IA sin revisarlo. Verificar que cumple los principios SOLID, los patrones de diseño y la Clean Architecture del proyecto.
13. **Pruebas del código generado:** todo código generado por IA debe estar cubierto por pruebas unitarias escritas (preferiblemente) por el equipo, como mecanismo de verificación.
14. **Control de versiones granular:** hacer commits frecuentes para poder rastrear qué partes del código fueron introducidas con asistencia de IA y qué modificaciones se hicieron posteriormente.
15. **Evitar dependencia ciega:** usar la IA como asistente, no como responsable del diseño arquitectónico. Las decisiones de arquitectura, patrones y principios deben ser tomadas por el equipo, con la IA como herramienta de implementación o consulta.
16. **Protección de información sensible:** no incluir en los prompts información confidencial como credenciales, datos de usuarios o claves API.
17. **Citación de fuentes generadas:** cuando la IA proporcione algoritmos, explicaciones o fragmentos no triviales, referenciarlos en los comentarios del código o en la documentación.

7.4 Consecuencias del uso no documentado

Advertencia

El uso de herramientas de IA sin documentación equivale a una falta de transparencia académica y será tratado con el mismo criterio que el plagio. Si durante la defensa el equipo no puede explicar el funcionamiento del código, los patrones aplicados o las decisiones de diseño, se procederá a descontar puntos de la calificación final, independientemente de la calidad aparente del código entregado.

8. Entregables

El equipo debe consignar los siguientes entregables en la fecha y mediante el canal indicado:

#	Entregable	Canal / Formato
1	URL del repositorio GitHub del juego (cliente)	Mensaje por Bandeja de Entrada M7
2	URL del repositorio GitHub del backend	Mensaje por Bandeja de Entrada M7
3	Diagrama de clases (imagen + archivo fuente)	Incluido en /docs del repo
4	Diagrama de capas Clean Architecture (imagen + fuente)	Incluido en /docs del repo
5	README completo en ambos repositorios	En GitHub (rama main)
6	Archivo AI_USAGE.md en ambos repositorios	En GitHub (rama main)
7	Ejecutable funcional del juego (Android o iOS)	Release en GitHub

9. Rúbrica de Evaluación — 20 Puntos

La calificación final del proyecto se distribuye en los siguientes criterios. Cada criterio se evalúa de forma integral considerando la calidad de la implementación, la coherencia con los principios del curso, la claridad de la documentación y la calidad y coherencia de la defensa individual de cada miembro del equipo.

Criterio	Descripción y nivel de logro esperado	Pts
1. Funcionalidad del Juego	El juego es completamente funcional: el tablero se renderiza correctamente, las flechas rotan, se mueve según el camino definido, y los flujos de victoria/derrota funcionan. Todos los niveles mínimos requeridos están implementados. La aplicación no presenta crashes durante la demostración. (Escala 4 pts: completo y sin errores. 3 pts: funcional con errores menores. 2 pts: funcional parcialmente. 1 pt: ejecuta pero con errores graves. 0 pts: no ejecuta.)	4 pts
2. Principios SOLID	Los principios SOLID están aplicados de forma correcta en el código. Cada principio aplicado está documentado en el README con un ejemplo concreto de código del proyecto. El equipo debe explicar la aplicación durante la defensa del proyecto. (Escala 2 pts: los 4 principios aplicados y documentados correctamente. 1 pt: al menos 3 principios aplicados. 0.5 pt: aplicación superficial o sin documentación. 0 pts: no se evidencia.)	2 pts
3. Patrones de Diseño	Se implementan patrones GoF distribuidos entre las tres categorías: creacionales, estructurales, comportamiento. (Escala 3 pts: 4+ patrones correctamente aplicados y defendidos. 2 pts: 2 patrones. 1 pt: 1 patrones o aplicación incorrecta. 0 pts: no se evidencia.)	3 pts
4. Arquitectura CLEAN	El proyecto implementa las cuatro capas de la <i>Clean Architecture</i> . La regla de dependencia se respeta estrictamente. El diagrama de capas es correcto y completo. Los casos de uso son independientes del framework. (Escala 3 pts: arquitectura correcta, diagrama completo, regla de dependencia respetada. 2 pts: arquitectura parcial o diagrama incompleto. 1 pt: capas identificadas pero con dependencias incorrectas. 0 pts: no se aplica.)	3 pts
5. AOP	Se implementan al menos 1 aspecto AOP que separan correctamente los cross-cutting concerns del código de negocio (negocio en este caso es juego). Los aspectos están documentados en el README con la estrategia de implementación basada en SOLID. (Escala 2 pts: 3+ aspectos correctos y documentados. 1 pt: 1-2 aspectos o implementación parcial. 0 pts: no se implementa.)	2 pts
6. Pruebas	Casos de prueba en capas de dominio y casos de uso. Se aplican pruebas unitarias (AAA), de integración y de contrato. El pipeline CI/CD ejecuta las pruebas automáticamente en cada PR. (Escala 2 pts: casos de pruebas sin fragilidad y cubre capa dominio y casos de uso, CI/CD configurado, nomenclatura correcta. 1 pt: pruebas parciales sin CI. 0 pts: sin pruebas.)	2 pts

7. Backend y API REST	El backend implementa todos los endpoints requeridos (por ejemplo: autenticación, progreso, leaderboard, niveles. Son solo ejemplos) con documentación Swagger. La comunicación cliente-servidor debe funcionar correctamente. (Escala 1 pt: backend completo y funcional. 0.5 pt: backend parcial. 0 pts: sin backend.)	1 pt
8. Documentación (README + Diagramas)	Ambos README están redactados en inglés o español y siguen la estructura mínima exigida. Los diagramas de clases y de capas son correctos, están incluidos en /docs y embebidos en el README. Los commits siguen Conventional Commits. (Escala 0.5 pt: documentación completa y profesional. 0 pts: documentación mínimo o sin documentación.)	0.5 pt
9. Documentación del Uso de IA	El archivo AI_USAGE.md está presente en ambos repositorios y cumple con los requisitos de la Sección 7: herramientas, registro de uso por tarea con prompts, modificaciones realizadas y evaluación crítica. La documentación es honesta y detallada. (Escala 0.5 pt: documentación completa y honesta. 0 pts: no existe o es insuficiente.)	0.5 pt
10. Defensa individual	Durante la sustentación, el equipo demuestra comprensión profunda de las decisiones técnicas tomadas. Cada miembro puede explicar los patrones, la arquitectura y el código, incluyendo el código generado con IA. Se evalúa la cohesión del equipo y la calidad de la presentación. (Escala 2 pts: estudiante demuestra comprensión sólida. 1 pt: comprensión parcial. 0 pts: no puede explicar su propio código.)	2 pts
TOTAL		20 pts

10. Integridad Académica y Trabajo en Equipo

Política de integridad académica

Queda estrictamente prohibido copiar código de otros grupos o de proyectos públicos de GitHub sin citar la fuente. La detección de plagio resultará en la anulación del proyecto para todos los integrantes del grupo.

El uso de IA está permitido bajo las condiciones de la Sección 7. El uso de IA sin documentación será tratado como falta de integridad académica.

Todos los miembros del grupo deben contribuir activamente. El historial de commits de GitHub será revisado para verificar la participación individual. Un miembro sin **commits significativos** podrá recibir una calificación individual reducida.

11. Referencias y Recursos Sugeridos

Libros y artículos

- Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

Recursos en línea

- Arrow Maze - Escape Puzzle en Google Play: <https://play.google.com/store/apps/details?id=com.arrows.escape>
- Conventional Commits: <https://www.conventionalcommits.org>
- Clean Architecture blog (Uncle Bob): <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>
- OWASP Best Practices: <https://owasp.org>
- GitHub Actions (CI/CD): <https://docs.github.com/en/actions>
- Documentación de Prompting para Claude: <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>
- OpenAI Prompt Engineering Guide: <https://platform.openai.com/docs/guides/prompt-engineering>